

Procesamiento de imágenes I



GRUPO 14

Año 2026

Integrantes:

- ***Aiup Agustin***
- ***Paoloni Franco***

Introducción

En este trabajo resolvimos dos problemas usando las técnicas vistas en las unidades 1 y 2 de la materia. En el primero implementamos una ecualización local de histograma y la usamos para encontrar detalles que estaban "escondidos" en una imagen. En el segundo armamos un script que valida automáticamente planillas de calificaciones a partir de una imagen, detectando la tabla, recortando cada campo y contando caracteres y palabras.

Todo el código está en Python, usando OpenCV, NumPy y Matplotlib. Intentamos apoyarnos lo más posible en los ejemplos de clase (por ejemplo, el cálculo del histograma y la CDF del script de transformaciones, y la detección de inicio y fin de "pulsos" del problema de las letras) y en las ayudas que da el enunciado.

Problema 1 — Ecualización local de histograma

Descripción del problema

La imagen `Imagen_con_detalles_escondidos.tif` tiene cinco cuadrados negros sobre un fondo gris claro. A simple vista los cuadrados parecen uniformes, pero adentro tienen detalles con intensidades apenas distintas a las del negro que los rodea. Lo primero que hicimos fue mirar el histograma:

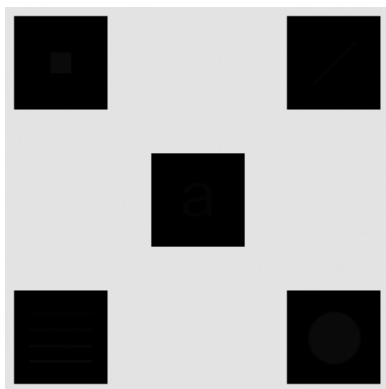


Figura 1. Imagen original.

a) Implementación de la función

Hicimos la función `ecualizacion_local(img, M, N)`, que recibe la imagen y el tamaño de la ventana $M \times N$. El procedimiento es el que plantea el enunciado:

- Agregamos un borde a la imagen con `cv2.copyMakeBorder(..., cv2.BORDER_REPLICATE)` para que la ventana siempre quede completa, incluso en los píxeles de los bordes.
- Recorremos la imagen píxel a píxel. En cada posición tomamos la ventana de $M \times N$ y calculamos su histograma con `cv2.calcHist`, lo normalizamos dividiendo por la cantidad de píxeles y hacemos la suma acumulada (`cumsum`) para obtener la función de distribución acumulada (CDF).
- Con esa CDF calculamos el nuevo valor sólo del píxel central: $s = 255 \cdot \text{cdf}(r)$, donde r es la intensidad original de ese píxel.

Problemas que encontramos

- **Bordes de la imagen:** sin agregar borde, la ventana se "sale" de la imagen en las primeras y últimas filas y columnas. Con BORDER_REPLICATE se replican los valores del borde y se evita que aparezcan valores artificiales (por ejemplo, ceros) que alteren el histograma.
- **Ventanas de tamaño par:** el enunciado permite cualquier M, N enteros positivos, pero con tamaños pares no hay un píxel central exacto. Lo resolvimos repartiendo el borde de forma desigual (arriba/izquierda $(M-1)//2$, abajo/derecha $M//2$), de forma que el "centro" queda corrido medio píxel hacia arriba a la izquierda.

b) Detalles ocultos

Aplicando la función con una ventana de 15x15 aparecen claramente los detalles de cada cuadrado:

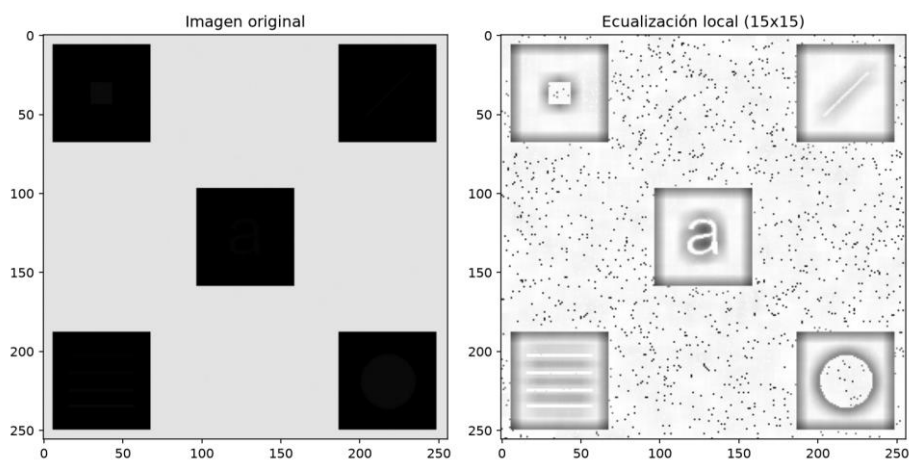


Figura 2. Imagen original y resultado de la ecualización local con ventana de 15x15.

c) Influencia del tamaño de la ventana

Probamos ventanas de 3x3, 7x7, 15x15, 31x31, 51x51 y 101x101:

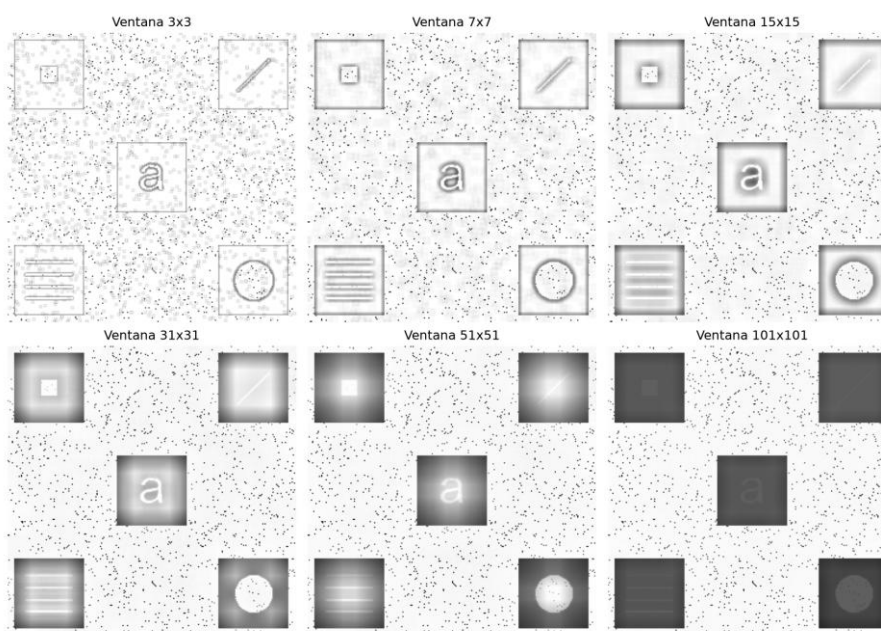


Figura 3. Resultado de la ecualización local para distintos tamaños de ventana.

- **Ventanas chicas (3x3, 7x7):** La ventana es más chica que los objetos, entonces cuando está completamente adentro de un detalle (o completamente afuera) el histograma es casi uniforme y no hay contraste. Sólo se realzan los bordes de los detalles, que se ven "huecos". Además, el ruido del fondo se amplifica muchísimo.
- **Ventanas medianas (15x15, 31x31):** Son las que mejor funcionan. La ventana es del orden del tamaño de los detalles, así que se ven completos y con buen contraste. Aparece un resplandor cerca de los bordes de los cuadrados, porque ahí la ventana ya incluye parte del fondo claro.
- **Ventanas grandes (51x51, 101x101):** El histograma de la ventana se parece cada vez más al de la imagen completa, y el resultado se acerca a una ecualización global: los detalles pierden contraste y con 101x101 casi no se distinguen.

Conclusiones del problema 1

La ecualización local permite resaltar detalles que tienen intensidades parecidas a las de su entorno y que se pierden con una ecualización global. El parámetro más importante es el tamaño de la ventana, que tiene que estar en proporción con el tamaño de lo que se quiere ver: si es muy chica sólo se ven bordes y ruido, y si es muy grande se pierde la información local. Para esta imagen, entre 15x15 y 31x31 obtuvimos el mejor resultado. Una desventaja de la técnica es que amplifica el ruido en las zonas uniformes.

Problema 2 — Validación de planillas de calificaciones

Descripción del problema

Tenemos cuatro imágenes de planillas (*grade_sheet_1.png* a *grade_sheet_4.png*), cada una con una tabla de 20 registros. Cada registro tiene Legajo, Nombre y Apellido, tres notas de parciales y la Condición Final. Hay que verificar que cada campo cumpla con estas restricciones:

Campo	Restricción
Legajo	8 caracteres, una sola palabra
Nombre y apellido	Al menos 2 palabras y no más de 12 caracteres
Parcial 1, 2 y 3	1 o 2 caracteres consecutivos (sin espacios)
Condición Final	Un único caracter

Es decir, no hace falta reconocer qué dice cada campo, sino contar caracteres y palabras. Dividimos el problema en tres partes:

- Detectar las líneas de la tabla para obtener las celdas.
- Recortar cada registro y cada uno de sus campos.
- Analizar cada campo: contar caracteres y palabras y verificar las restricciones.

a) Validación de los campos

Paso 1: umbralado

Como el texto y las líneas son negros y el fondo de la tabla es blanco, umbralizamos con $\text{img_th} = \text{img} < 150$. Así quedan en True los píxeles de las líneas y del texto.

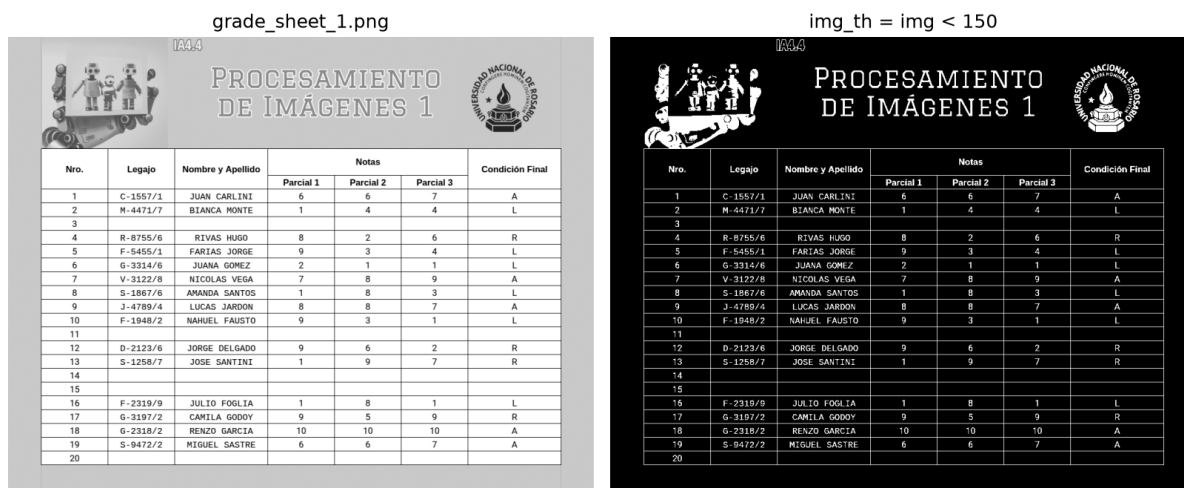


Figura 4. Planilla original y planilla umbralizada.

Paso 2: detección de las líneas de la tabla

Siguiendo la ayuda del enunciado, sumamos los píxeles de cada fila ($\text{np.sum}(\text{img_th}, 1)$) y de cada columna ($\text{np.sum}(\text{img_th}, 0)$).

Para encontrar dónde empieza y termina cada línea: el vector booleano $\text{img_rows} > \text{th_row}$ es una señal de "pulsos", y con $\text{np.diff} + \text{np.argwhere}$ obtenemos el inicio y el fin de cada pulso. Lo armamos en la función `inicio_fin(v)`, que después volvimos a usar para contar palabras.

Paso 3: recorte de registros y campos

Con las líneas detectadas, cada celda queda entre el fin de una línea y el inicio de la siguiente. Sumando 1 al fin de la línea nos aseguramos de no incluir píxeles de la línea dentro de la celda. La primera columna (Nro.) no se analiza. Cada registro lo guardamos en un diccionario con su número, sus coordenadas y las sub-imágenes de sus seis campos, de la misma forma que en el problema de las letras.

Paso 4: conteo de caracteres y palabras

Para contar caracteres usamos `cv2.connectedComponentsWithStats`: cada caracter es una componente conectada. Filtramos las componentes de área muy chica con `stats[:, -1] > th_area` para descartar posibles restos de las líneas. Tuvimos que tener cuidado con el valor de `th_area`: un guion ("-") tiene un área de sólo 4 píxeles, así que un umbral alto lo eliminaba y los legajos daban mal. Terminamos usando `th_area = 2`.

Para contar palabras analizamos las columnas de la celda con `celda.any(axis=0)`, igual que en el problema de las letras: cada caracter genera un pulso y los espacios entre palabras son tramos largos sin texto. Medimos las separaciones en las planillas y entre letras de una misma palabra hay como máximo 4 columnas vacías, mientras que un espacio deja 11 o más. Por eso tomamos como espacio una separación de más de 6 columnas.

Casos particulares

Con esta forma de contar se resuelven bien los casos complicados que aparecen en las planillas:

- Notas como "AUS", "100" o "1.0": tienen 3 caracteres (el punto también es una componente), así que dan MAL.
- Condiciones como "RR", "LL", "Rec" o "REC": tienen más de un caracter y dan MAL.
- Legajos con un espacio en el medio, como "R-47 89/4": tienen 8 caracteres pero 2 palabras, así que dan MAL.
- "AMANDASANTOS" da MAL (una sola palabra) y "JUAN CRUZ GAGO" da OK (3 palabras, 12 caracteres).
- Los campos vacíos tienen 0 caracteres y no cumplen ninguna restricción.

Las filas completamente vacías (por ejemplo, de la 18 a la 20 en algunas planillas) decidimos no informarlas, porque no son registros cargados. Igual conservamos el número de fila como identificador. Un ejemplo de la salida por terminal:

```
$ python problema2.py
> Registro 1:
> Legajo: OK
> Nombre y apellido: MAL
> Parcial 1: OK
> Parcial 2: OK
> Parcial 3: OK
> Condición Final: OK
>
> Registro 2:
> Legajo: OK
> Nombre y apellido: OK
> Parcial 1: OK
> Parcial 2: OK
> Parcial 3: OK
> Condición Final: OK
>
...

```

Figura 5. Salida por terminal para los primeros registros de grade_sheet_2.png.

b) Imagen de alumnos que no aprobaron

Para este punto hay que saber si la Condición Final es "L" (libre) o "R" (recupera), o sea reconocer la letra. Como sólo nos interesa separar L y R del resto, no hicimos un reconocedor general, sino que miramos la forma de la letra, que obtenemos con `connectedComponentsWithStats` recortando su caja contenedora:

- **L:** la columna izquierda está completa, no tiene agujeros y la parte superior derecha está vacía.
- **R:** la columna izquierda está completa, tiene un agujero en la mitad de arriba y la "pata" llega a la esquina inferior derecha.
- **A:** la columna izquierda no está completa, así que no se confunde con las otras dos.

Para contar agujeros volvimos a usar componentes conectadas, pero sobre el fondo de la letra: le agregamos un borde con `copyMakeBorder` para que todo el exterior quede en una sola componente, y cualquier otra componente del fondo es un agujero.

Con eso, para cada registro cargado correctamente (los seis campos en OK) con condición L o R, recortamos el campo Nombre y Apellido de la imagen original y le agregamos a la izquierda un recuadro de color: naranja con una R para los que recuperan y rojo con una L para los libres. Todas las filas se apilan en una única imagen.

L	BIANCA MONTE
R	RIVAS HUGO
L	FARIAS JORGE
L	JUANA GOMEZ
L	AMANDA SANTOS
L	NAHUEL FAUSTO
R	JORGE DELGADO
R	JOSE SANTINI
L	JULIO FOGLIA
R	CAMILA GODOY

Figura 6. Imagen de salida para grade_sheet_1.png.

c) Archivo CSV

Usando el módulo `csv` de Python generamos un archivo por planilla, donde cada fila es un registro con su ID (el número de fila en la planilla) y el resultado OK/MAL de cada campo, en el orden que pide el enunciado:

ID	Legajo	Nombre y apellido	Parcial 1	Parcial 2	Parcial 3	Condición Final
1	OK	MAL	OK	OK	OK	OK
2	OK	OK	OK	OK	OK	OK
3	MAL	MAL	OK	OK	OK	OK
4	MAL	OK	OK	OK	OK	OK
5	OK	OK	OK	OK	OK	OK
6	OK	OK	OK	OK	MAL	OK
7	MAL	OK	OK	OK	OK	MAL
...	...	...	...	...	...	...

d) Procesamiento de las cuatro planillas

Por último, aplicamos todo el algoritmo en un ciclo `for` sobre las cuatro planillas. Para cada una se imprime la validación, se genera *no_aprobados_<id>.png* y *resultados_<id>.csv*. Verificamos los resultados campo por campo contra las imágenes y coinciden en todos los casos.

Planilla	Registros con datos	Cargados correctamente	No aprobados
grade_sheet_1	15	15	10 (6 L, 4 R)
grade_sheet_2	17	3	3 (3 L)
grade_sheet_3	17	0	Ninguno
grade_sheet_4	17	6	4 (2 L, 2 R)

Conclusiones del problema 2

Pudimos validar las planillas sin reconocer los caracteres, sólo con umbralado, proyecciones por filas y columnas y componentes conectadas. La parte más delicada fue elegir los umbrales: usar uno relativo para las líneas hizo que el mismo código funcione en planillas de distinto tamaño, y los umbrales de área y de espacio los tuvimos que ajustar midiendo los caracteres reales (sobre todo por el guion y por las letras que se tocan).

La principal limitación es que los umbrales dependen de la fuente y del tamaño del texto: con letra manuscrita o con otra resolución habría que volver a ajustarlos. Lo mismo pasa con la clasificación de L y R, que se basa en la forma de esas letras con esta fuente y no serviría para reconocer cualquier caracter.